TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos

Profesores:

Ing. Jorge Arun Zambrano Cedeño, Mg

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: WLADIMIR OMAR HARO RODRIGUEZ

PARALELO: A

SEMESTRE: 5 NIVEL

ACTIVIDAD: Actividad #1 – Estudio de caso: Sistema de Control de Rescate y Adopción Animal

PERIODO SEPTIEMBRE 2026 - ENERO 2027

1. ANÁLISIS DEL PROBLEMA

El refugio necesita un sistema de información que permita gestionar los casos de animales rescatados y realizar el seguimiento de cada caso hasta su posible adopción. Actualmente, el manejo de esta información puede realizarse de manera manual, lo que puede ocasionar pérdida de información, dificultad para conocer el estado de los animales y problemas para asignar responsables.

El sistema permitirá registrar los animales que ingresan al refugio, almacenar sus datos, identificar su especie, asignar un voluntario o veterinario responsable y controlar el estado de cada caso.

Los principales estados que tendrá un animal dentro del sistema serán: **Rescatado, En Tratamiento y En Adopción**. De esta manera, se podrá conocer el proceso en el que se encuentra cada animal y realizar un mejor seguimiento.

El sistema estará orientado al trabajo de ciudadanos o reportantes, voluntarios, veterinarios y administradores del refugio.

2. ACTORES PRINCIPALES

2.1 Ciudadano/Reportante

Es la persona que comunica o registra información sobre un animal que necesita ser rescatado. Puede proporcionar datos básicos sobre la ubicación, especie y situación del animal.

2.2 Voluntario/Veterinario

Es el responsable de atender al animal después del rescate. Puede registrar información relacionada con la atención, tratamiento y evolución del animal, además de actualizar el estado del caso.

2.3 Administrador del Refugio

Es el encargado de administrar y supervisar los casos registrados en el sistema. Puede asignar responsables, consultar información, actualizar estados y gestionar el proceso de adopción.

3. REQUERIMIENTOS FUNCIONALES

Los requerimientos funcionales definidos para el sistema son los siguientes:

1. **Crear caso de rescate:** permitir registrar un nuevo caso de un animal que necesita ser rescatado.
2. **Registrar datos del animal:** almacenar información como especie, características y datos relacionados con el rescate.
3. **Categorizar según especie:** identificar si el animal corresponde a un perro, gato o fauna silvestre.
4. **Registrar perros, gatos y fauna silvestre:** permitir gestionar diferentes tipos de animales.
5. **Asignar voluntario o veterinario:** permitir al administrador asignar una persona responsable para la atención del animal.
6. **Actualizar estado del caso:** permitir cambiar el estado del animal entre Rescatado, En Tratamiento y En Adopción.
7. **Consultar casos registrados:** permitir consultar la información y el estado de los animales registrados.
8. **Gestionar adopción:** permitir administrar el proceso relacionado con la adopción de los animales.
9. **Generar notificaciones:** informar a los responsables cuando se registre un nuevo caso o cuando cambie el estado de un animal.

4. REQUERIMIENTOS NO FUNCIONALES

4.1 Escalabilidad

El sistema debe permitir incorporar nuevos tipos de animales y nuevas funcionalidades en el futuro sin tener que modificar completamente su estructura.

4.2 Mantenibilidad

El sistema debe estar organizado de manera que sus componentes puedan ser modificados o actualizados con facilidad.

4.3 Trazabilidad

Los cambios realizados durante el desarrollo deben poder identificarse mediante el sistema de control de versiones, permitiendo conocer las modificaciones realizadas en el proyecto.

4.4 Reutilización

Los componentes del sistema deben diseñarse de manera que puedan ser reutilizados en diferentes partes de la aplicación, evitando repetir código innecesariamente.

5. CONTROL DE VERSIONES CON GITFLOW

Para el desarrollo del sistema se utilizará Git y GitHub como herramientas de control de versiones. Se aplicará el flujo de trabajo Gitflow para organizar las diferentes etapas del desarrollo.

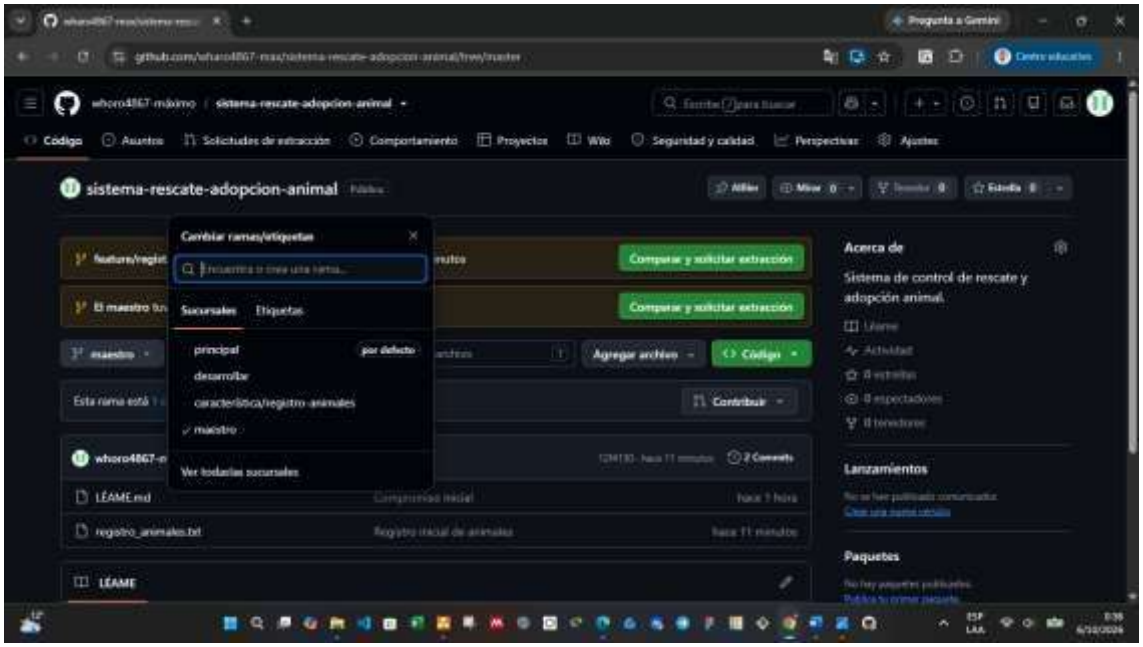
Las ramas principales serán:

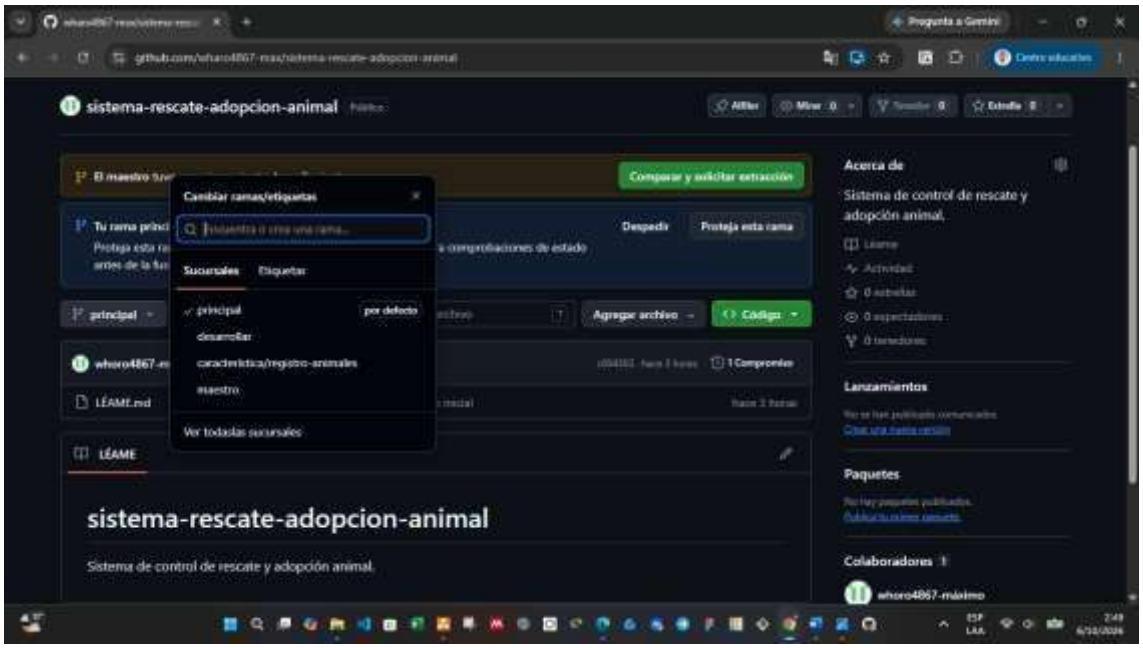
- **master:** contendrá las versiones estables del sistema.
- **develop:** contendrá los avances integrados antes de pasar a una versión estable.
- **feature:** se utilizará para desarrollar funcionalidades específicas del sistema.

Algunos ejemplos de ramas de funcionalidades son:

- **feature/registro-animales**
- **feature/asignacion-responsables**
- **feature/notificaciones**

Este esquema permitirá organizar el desarrollo, controlar los cambios y mantener un historial de las modificaciones realizadas en el proyecto.





6. PATRONES DE DISEÑO

Para el desarrollo del Sistema de Control de Rescate y Adopción Animal se utilizarán diferentes patrones de diseño con el propósito de organizar mejor el sistema, facilitar su mantenimiento y permitir que sus componentes puedan crecer de manera ordenada.

Los patrones seleccionados son **Singleton, Observer, Factory Method y MVC**.

6.1 Singleton

El patrón Singleton permitirá controlar una instancia única para componentes que deben ser centralizados, como la configuración general del sistema o la conexión con la base de datos.

Su utilización evita crear múltiples instancias innecesarias del mismo componente y permite que los diferentes actores del sistema puedan acceder a una gestión centralizada de la información.

Problema que soluciona: evita la creación de varias instancias de un componente que debe ser único dentro del sistema.

6.2 Observer

El patrón Observer se utilizará para implementar el sistema de notificaciones.

Cuando se registre un nuevo caso de rescate o se produzca un cambio en el estado de un animal, los usuarios responsables, como voluntarios o veterinarios, podrán recibir una notificación automáticamente.

Problema que soluciona: evita que los responsables tengan que consultar constantemente el sistema para conocer si existe un nuevo caso o un cambio en el estado de un animal.

6.3 Factory Method

El patrón Factory Method permitirá crear diferentes tipos de objetos de animales de acuerdo con la especie registrada.

Por ejemplo, el sistema podrá crear objetos correspondientes a:

- Perro.
- Gato.
- Fauna silvestre.

Cada tipo de animal podrá tener características específicas sin afectar el funcionamiento general del sistema.

Problema que soluciona: facilita la creación de diferentes tipos de objetos y permite incorporar nuevas especies posteriormente sin modificar toda la estructura del sistema.

6.4 MVC

El patrón MVC (Modelo-Vista-Controlador) permitirá separar las diferentes responsabilidades de la aplicación.

- **Modelo:** administra los datos y la lógica relacionada con la información del sistema.

- **Vista:** presenta la información y la interfaz que utilizarán los usuarios.
- **Controlador:** recibe las acciones del usuario y coordina la comunicación entre la Vista y el Modelo.

Esta separación permite realizar cambios en una parte del sistema sin afectar directamente las demás partes.

Problema que soluciona: facilita el mantenimiento, organización y modificación del sistema al separar la presentación, la lógica y los datos.

7. PRINCIPIOS SOLID

El diseño del sistema tendrá en cuenta los principios SOLID para mejorar la organización y mantenimiento del código.

Se buscará que cada clase tenga una responsabilidad específica y que los diferentes componentes puedan modificarse o ampliarse sin afectar innecesariamente al resto del sistema.

Por ejemplo, las funciones relacionadas con los animales, usuarios, notificaciones y adopciones deberán mantenerse organizadas en componentes independientes. Esto permitirá que el sistema sea más fácil de mantener y ampliar.

La aplicación de estos principios también ayudará a reducir la dependencia entre componentes y facilitará la reutilización del código.

8. DIAGRAMA DE CASOS DE USO

En esta sección se incluirá el **diagrama de casos de uso realizado en Draw.io**.

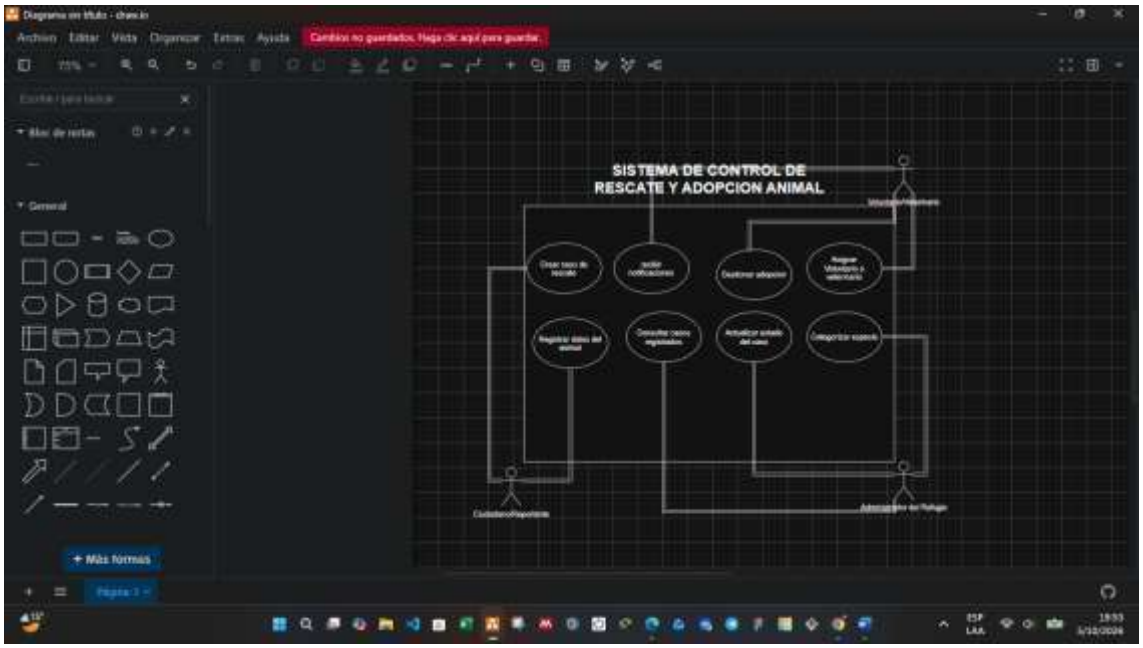
El diagrama representará la interacción de los siguientes actores con el sistema:

- Ciudadano/Reportante.

- Voluntario/Veterinario.
- Administrador del Refugio. Los principales casos de uso serán:

- Crear caso de rescate.
- Registrar datos del animal.
- Categorizar especie.
- Asignar voluntario o veterinario.
- Actualizar estado del caso.

- Consultar casos registrados.
- Gestionar adopción.
- Recibir notificaciones.



9. DIAGRAMA DE CLASES UML

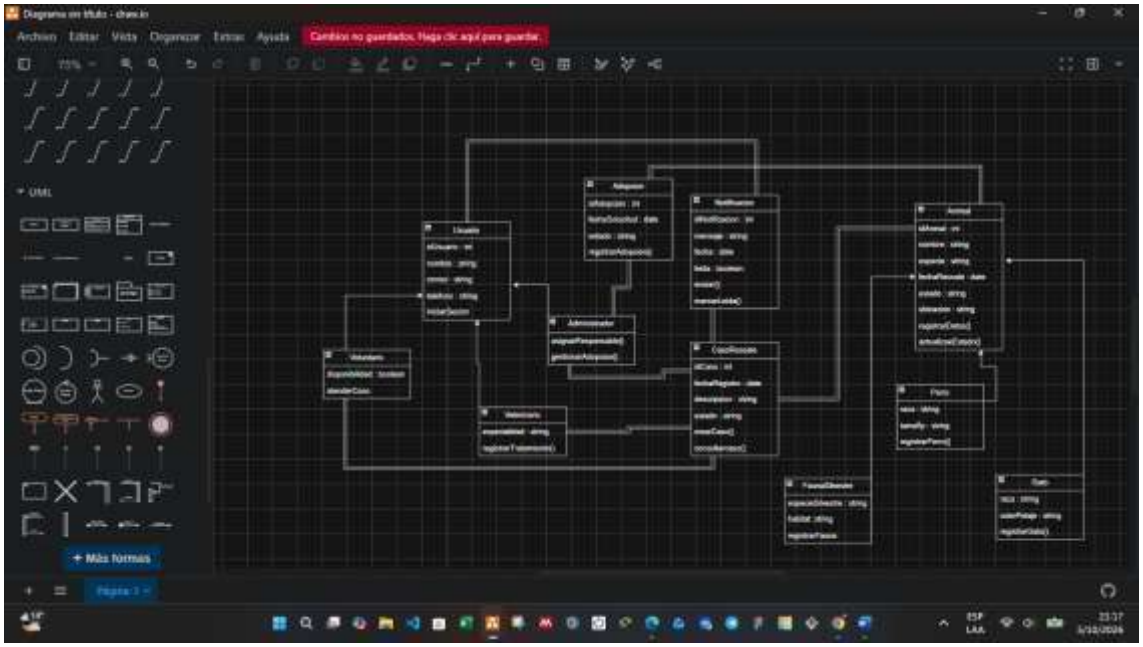
En esta sección se incluirá el **diagrama de clases UML realizado en Draw.io**.

El diagrama representará las principales clases del sistema, sus atributos, métodos y relaciones, tomando en cuenta los requerimientos definidos y los principios SOLID.

Entre las clases principales estarán:

- Animal.
- Perro.
- Gato.
- FaunaSilvestre.

- CasoRescate.
- Usuario.
- Voluntario.
- Veterinario.
- Administrador.
- Adopción.
- Notificación.



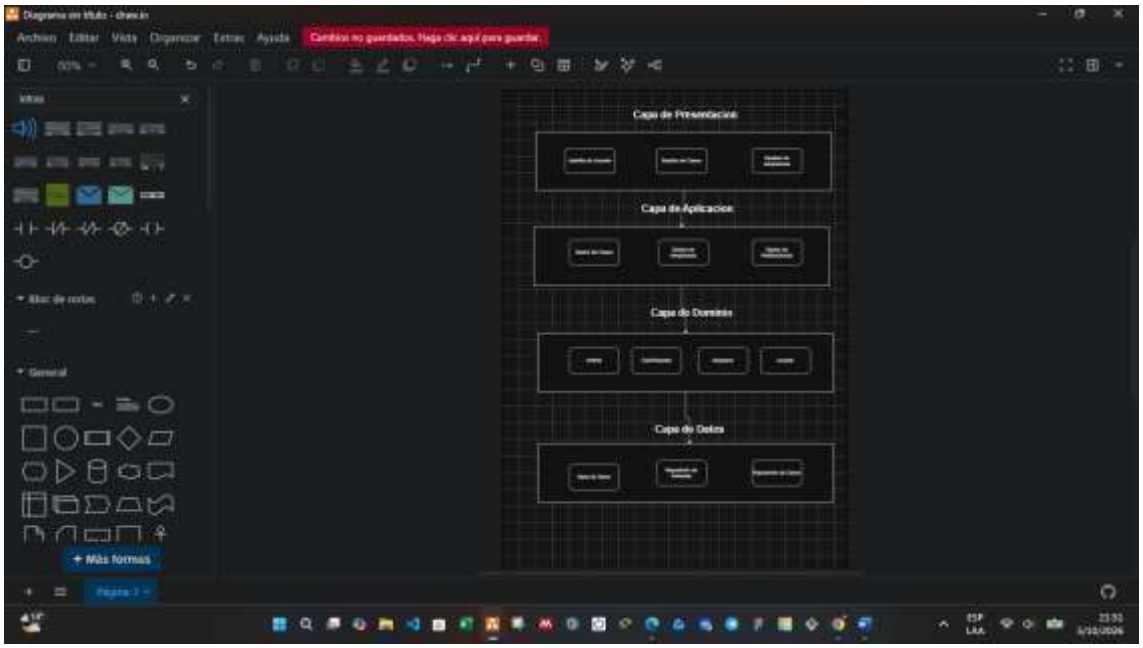
10. ARQUITECTURA GENERAL DEL SISTEMA

La arquitectura general representará la organización de los principales componentes del sistema y la comunicación entre ellos.

Se considerarán las siguientes partes:

- **Interfaz de usuario:** permite que los ciudadanos, voluntarios, veterinarios y administradores utilicen el sistema.
- **Controladores:** reciben las solicitudes de los usuarios y coordinan las operaciones.
- **Lógica del sistema:** contiene las reglas relacionadas con rescates, animales, tratamientos, adopciones y notificaciones.
- **Base de datos:** almacena la información de los animales, usuarios, casos y adopciones.

- **Sistema de control de versiones:** Git y GitHub permitirán controlar los cambios realizados durante el desarrollo.



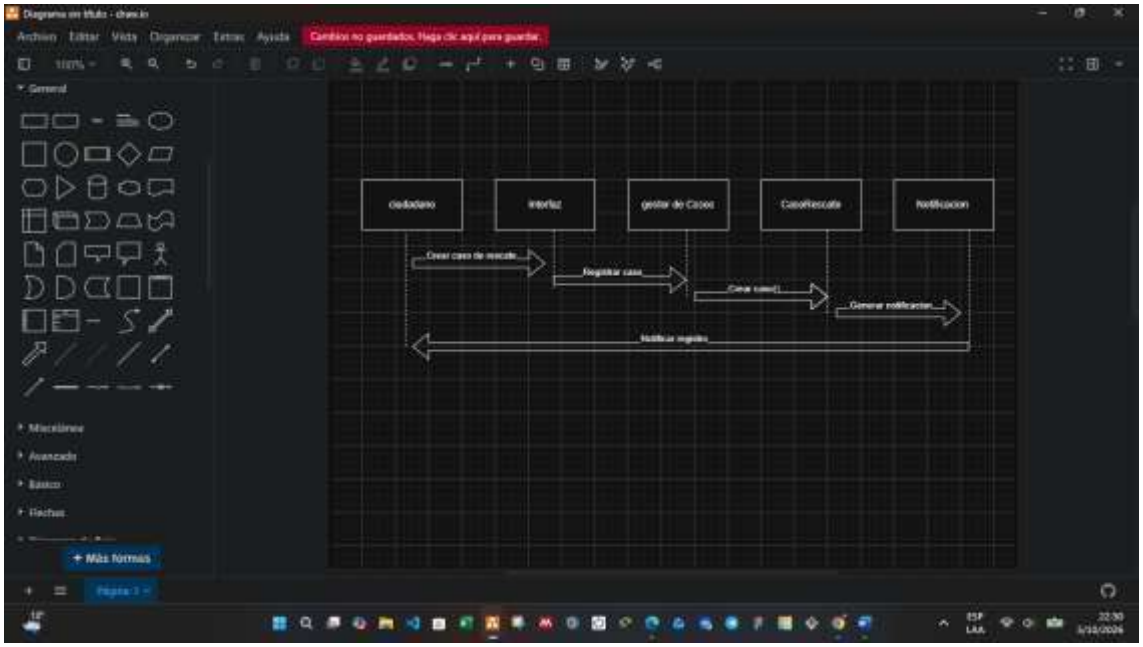
11. DIAGRAMA DE SECUENCIA

El diagrama de secuencia representará el proceso de registro de un nuevo caso de rescate.

El proceso iniciará cuando el Ciudadano/Reportante registre la información del animal. El sistema procesará los datos, creará el caso de rescate y notificará al responsable correspondiente.

La secuencia principal será:

1. El Ciudadano/Reportante registra un caso.
2. La Vista envía la información al Controlador.
3. El Controlador valida los datos.
4. El sistema registra el caso.
5. Se almacena la información en la base de datos.
6. El sistema genera una notificación.
7. El Voluntario/Veterinario recibe la notificación.



12. IMPLEMENTACIÓN DE LOS PATRONES DE DISEÑO

Los patrones seleccionados se relacionarán con diferentes partes del sistema.

- **Singleton:** utilizado para administrar componentes centralizados, como la conexión o configuración general.
- **Observer:** utilizado para las notificaciones relacionadas con nuevos casos y cambios de estado.
- **Factory Method:** utilizado para crear objetos correspondientes a las diferentes especies de animales.
- **MVC:** utilizado para separar la interfaz, el procesamiento de las solicitudes y la gestión de los datos.

La combinación de estos patrones permitirá obtener una estructura organizada, mantenible y preparada para futuras ampliaciones.

13. FLUJO DE TRABAJO GITFLOW

El desarrollo del proyecto seguirá el siguiente flujo:

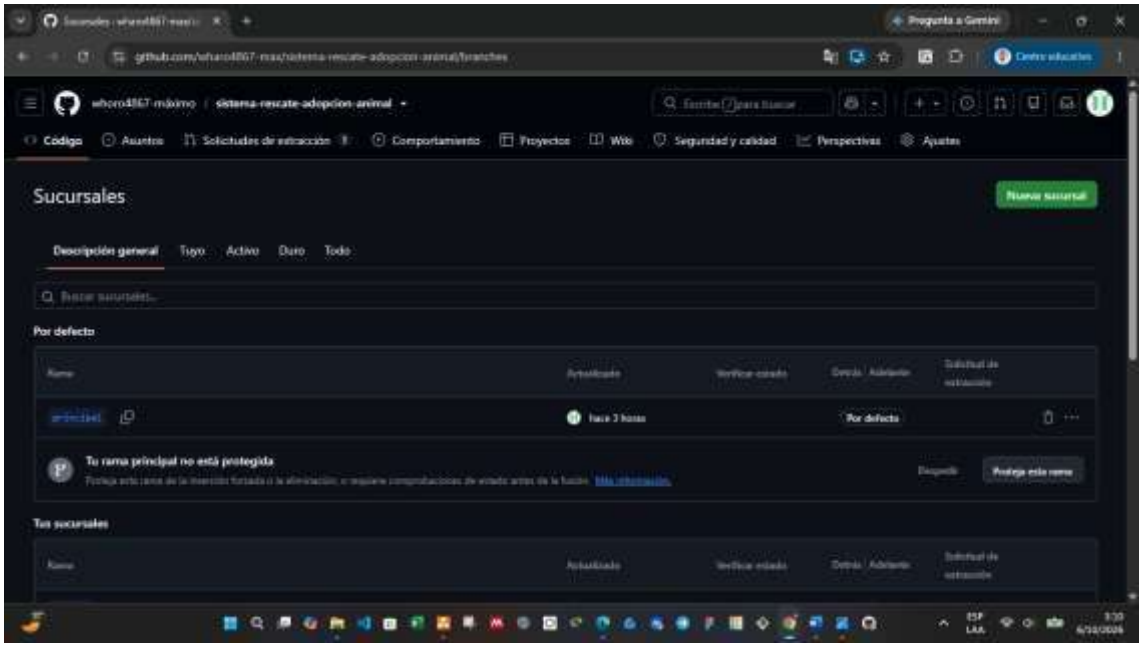
master → develop → feature

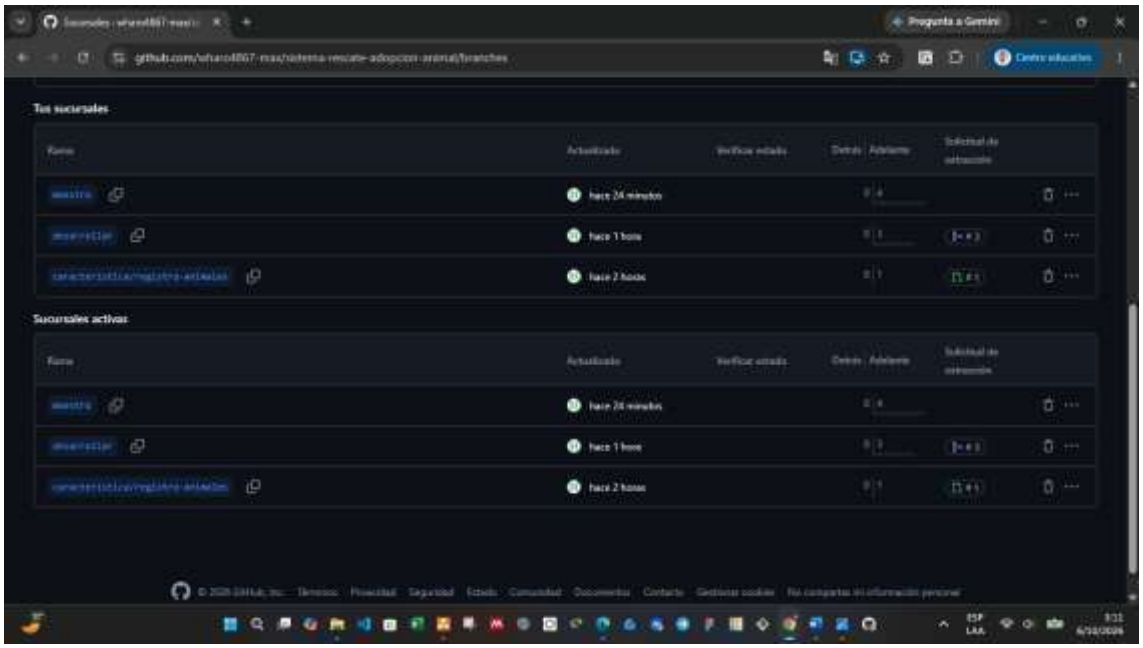
La rama **master** mantendrá las versiones estables. La rama **develop** integrará los avances realizados y las ramas **feature** se utilizarán para desarrollar funcionalidades específicas.

Ejemplo:

develop → feature/registro-animales → develop → master

Este flujo permitirá mantener un control organizado de las versiones del sistema.





14. CONCLUSIONES

El desarrollo del Sistema de Control de Rescate y Adopción Animal permitirá organizar la información relacionada con los animales rescatados y realizar un seguimiento de cada caso durante las diferentes etapas del proceso.

El análisis de actores y requerimientos permitió identificar las principales funciones que debe cumplir el sistema. Además, la utilización de patrones como Singleton, Observer, Factory Method y MVC permitirá organizar mejor el funcionamiento de la aplicación.

La aplicación de principios SOLID contribuirá a desarrollar un sistema más organizado y fácil de mantener, mientras que Git y Gitflow permitirán controlar los cambios realizados durante el desarrollo.

Finalmente, la propuesta permitirá contar con una base estructurada para desarrollar un sistema que facilite la gestión de rescates, tratamientos y procesos de adopción de animales.

15. EVIDENCIAS

En esta sección se incluirán las evidencias correspondientes al desarrollo de la actividad:

- Captura del diagrama de casos de uso.
- Captura del diagrama de clases UML.
- Captura de la arquitectura general.
- Captura del diagrama de secuencia.
- Capturas del repositorio de GitHub.
- Capturas de las ramas Gitflow.
- Capturas de los commits realizados.

16. ENLACE DEL REPOSITORIO DE GITHUB

Repositorio público:

https://github.com/wharo4867-max/proyecto-git/tree/principal

https://github.com/wharo4867-max/sistema-rescate-adopcion-animal/tree/develop

Bibliografía

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://gitscm.com/book/en/v2

GitHub. (s. f.). *Branches*. GitHub Docs. https://docs.github.com/en/pullrequests/reference/branches

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. https://www.omg.org/spec/UML/2.5.1/

Driessen, V. (2010). *A successful Git branching model*. nvie. https://nvie.com/posts/asuccessful-git-branching-model/

